Programming with Python

BASIC CONCEPTS 1

Lesson Objectives

- Introduction to Python
- Variables
- Operators
- Loops
- Conditional statements
- Strings

Introduction to Python

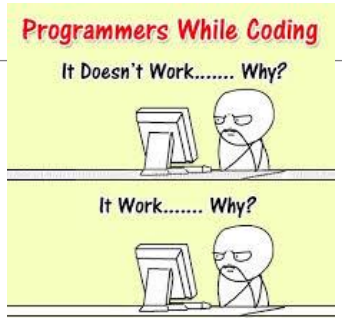
SECTION 1

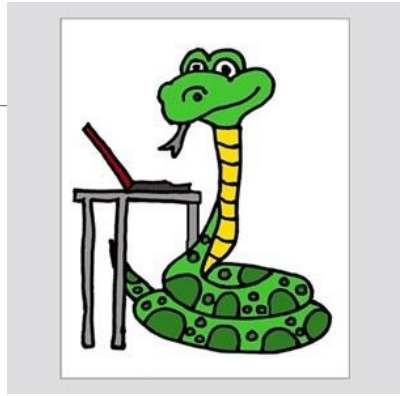
Introduction to Python

- ***Python*** *is a widely-used, interpreted, object-oriented, and high-level programming language with dynamic semantics, used for general-purpose programming.*
- ***Python*** *was created by Guido van Rossum, born in 1956 in Haarlem, the Netherlands*

Introduction to Python

- *What makes Python special?*
  - ✔ *it's easy to learn*
  - ✔ *it's easy to teach*
  - ✔ *it's easy to use*
  - ✔ *it's easy to understand*
  - ✔ *it's easy to obtain, install and deploy*





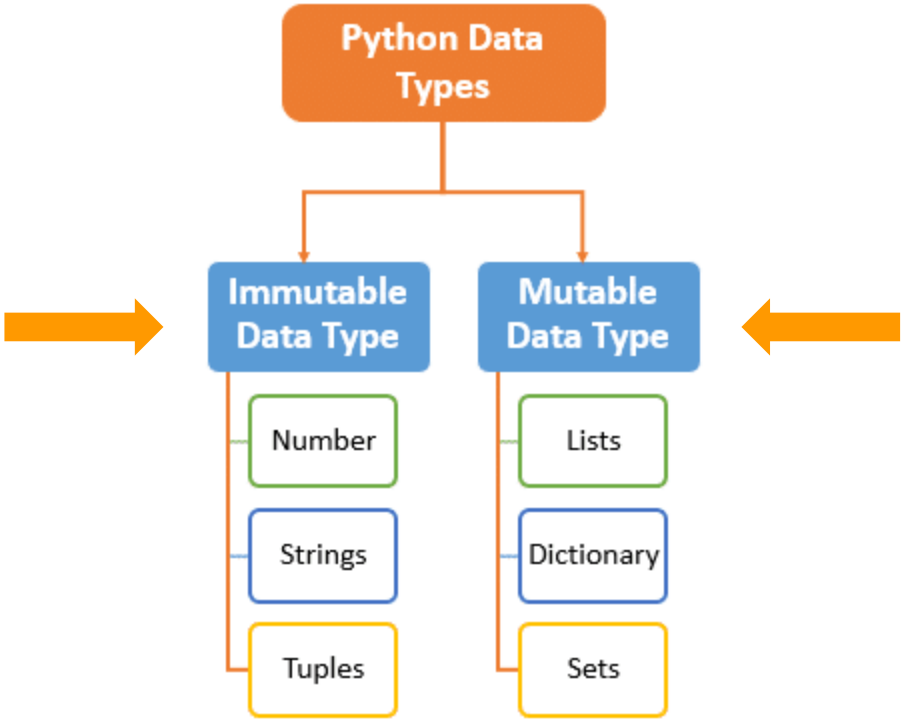
Introduction to Python

- *Drawbacks of Python*
  - ✔ *it's not a speed demon*
  - ✔ *debugging Python's code can be more difficult than with other languages*
- *Some niches where Python is rarely seen*
  - ✔ *low-level programming*
  - ✔ *applications for mobile devices*

Data types & Variables

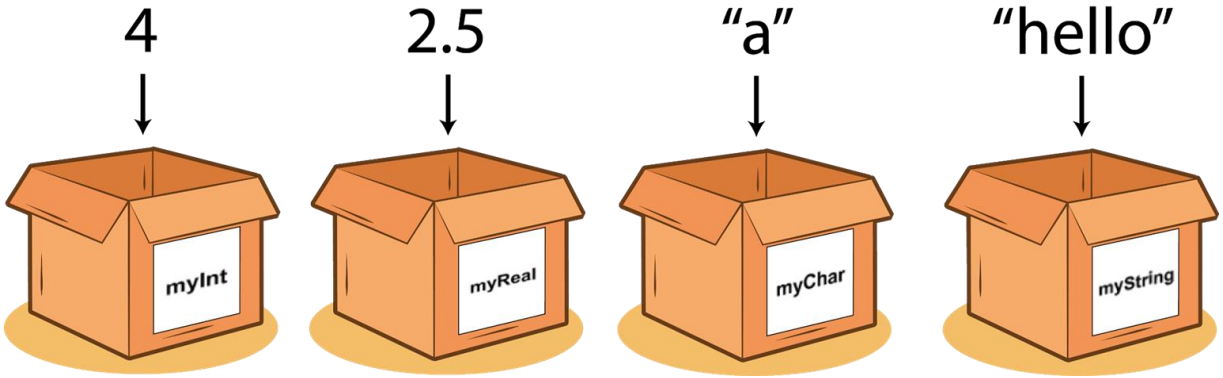
SECTION 2

Data types


Python Data Types
Immutable Data Type
Mutable Data Type
Number
Strings
Tuples
Lists
Dictionary
Sets

Variable

- A variable is a named location reserved to store values in the memory.
- A variable is created or initialized automatically when you assign a value to it for the first time



Variable Naming Rules

- *A legal identifier name must be:*
  - ✔ *a non-empty sequence of characters,*
  - ✔ *must begin with the underscore(_), or a letter,*
  - ✔ *cannot be a Python keyword.*

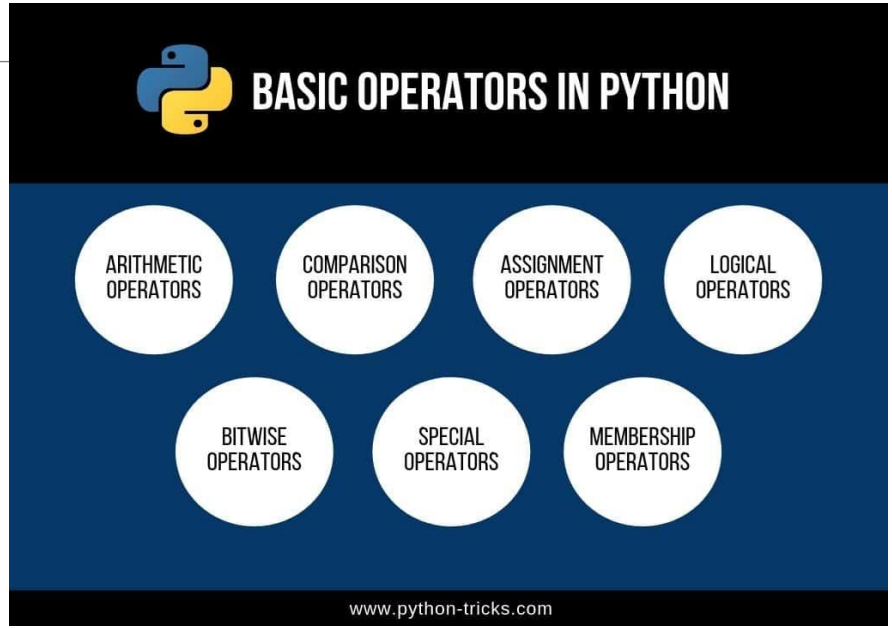
Keywords in Python				
False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

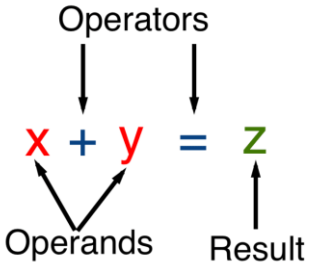
Operators

SECTION 3

Operators - What is an operator?

- An operator is a symbol of the programming language, which is able to operate on the values.
- Basic Python operators: +, -, *, /, //, %, **





Operators - Arithmetic operators

- **MULTIPLICATION**: An * (asterisk) sign is a multiplication operator.
- **DIVISION**: A / (slash) sign is a divisional operator.
- The value in front of the slash is a dividend, the value behind the slash, a divisor.
- **Remember**:
  - The result produced by the division operator is always a float, regardless of whether or not the result seems to be a float at first glance: 1 / 2, or if it looks like a pure integer: 2 / 1.

Operators - Arithmetic operators

- **INTEGER DIVISION**:
- It differs from the standard / operator in two details:
  - ✔ its result lacks the fractional part - it's absent (for integers), or is always equal to zero (for floats); this means that the results are always rounded;
  - ✔ it conforms to the integer vs. float rule.
- **Remember**: rounding goes toward the lesser integer value (floor division)

Operators - Arithmetic operators

- **EXPONENTIATION**: A ** (double asterisk) sign is an exponentiation (power) operator. Its left argument is the base, its right, the exponent
- **Remember**:
  - ✔ when both ** arguments are integers, the result is an integer, too;
  - ✔ when at least one ** argument is a float, the result is a float, too.

Operators - Arithmetic operators

- **REMAINDER (MODULO):** The result of the % operator is a remainder left after the integer division

  - `14 // 4` gives `3` → this is the integer **quotient**;
  - `3 * 4` gives `12` → as a result of **quotient and divisor multiplication**;
  - `14 - 12` gives `2` → this is the **remainder**.

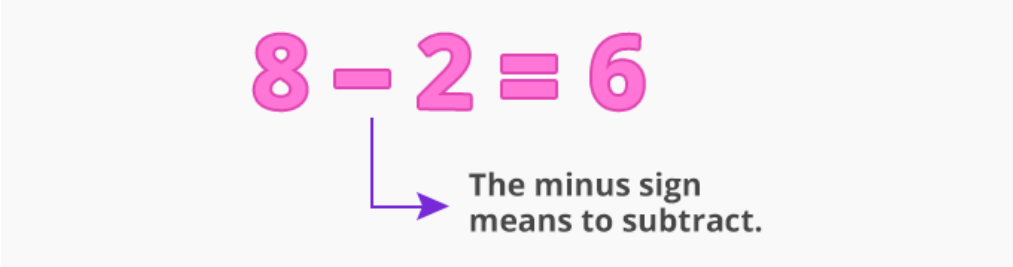
- **ADDITION**: The addition operator is the + (plus) sign, which is fully in line with mathematical standards.
- **SUBTRACTION**: The subtraction operator is obviously the - (minus) sign, although you should note that this operator also has another meaning - it can change the sign of a number.

Operators - Unary vs binary operators

- **UNARY**: Expects only one operand on the right side of the operator.

-1

- **BINARY**: Expects two operanc ft and right sides of the operator.



Operators - Comparison Operators

Operator	Description	Example
`==`	returns if operands' values are equal, and `False` otherwise	`x == y # False` `x == z # True`
`!=`	returns `True` if operands' values are not equal, and `False` otherwise	`x != y # True` `x != z # False`
`>`	`True` if the left operand's value is greater than the right operand's value, and `False` otherwise	`x > y # False` `y > z # True`
`<`	`True` if the left operand's value is less than the right operand's value, and `False` otherwise	`x < y # True` `y < z # False`

`≥`	`True` if the left operand's value is greater than or equal to the right operand's value, and `False` otherwise	`x >= y # False`
		`x >= z # True`
		`y >= z # True`

`≤`	`True` if the left operand's value is less than or equal to the right operand's value, and `False` otherwise	`x <= y # True`
		`x <= z # True`
		`y <= z # False`

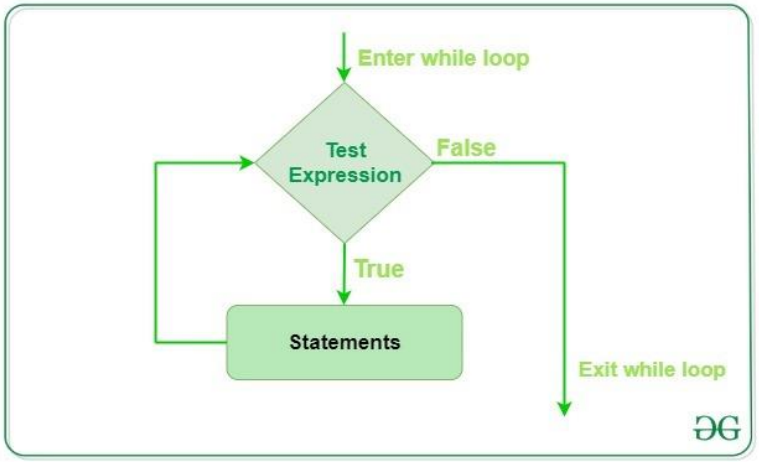
Operators - The hierarchy of priorities

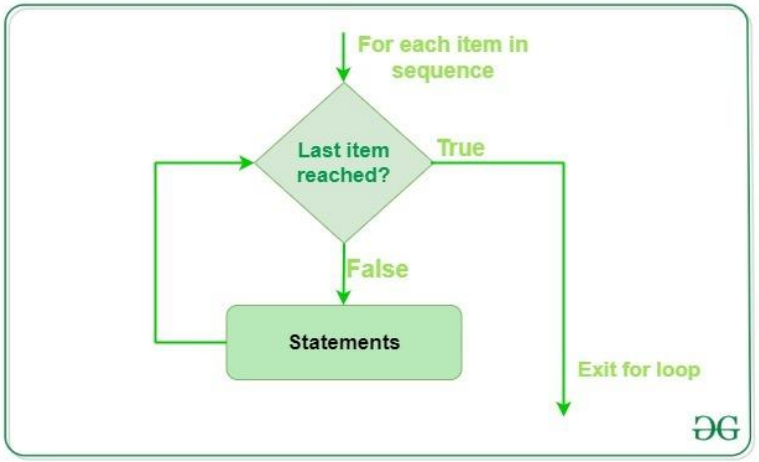
Priority	Operator	
1	`+`, `-`	unary
2	`**`	
3	`*`, `/`, `//`, `%`	
4	`+`, `-`	binary
5	`<`, `<=`, `>`, `>=`	
6	`==`, `!=`	

Loops

SECTION 4

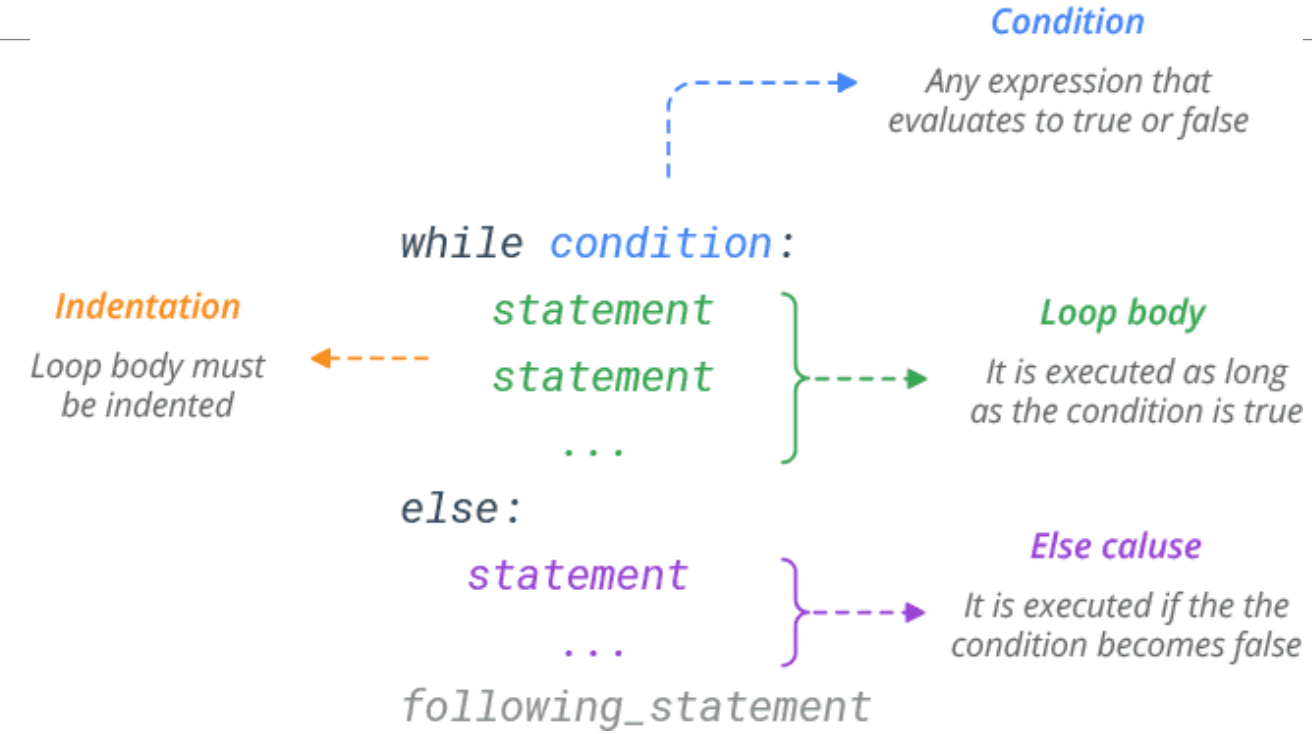
Loops


Enter while loop
Test Expression
False
True
Statements
Exit while loop


For each item in sequence
Last item reached?
True
False
Statements
Exit for loop

Loops - While

- The **while** loop

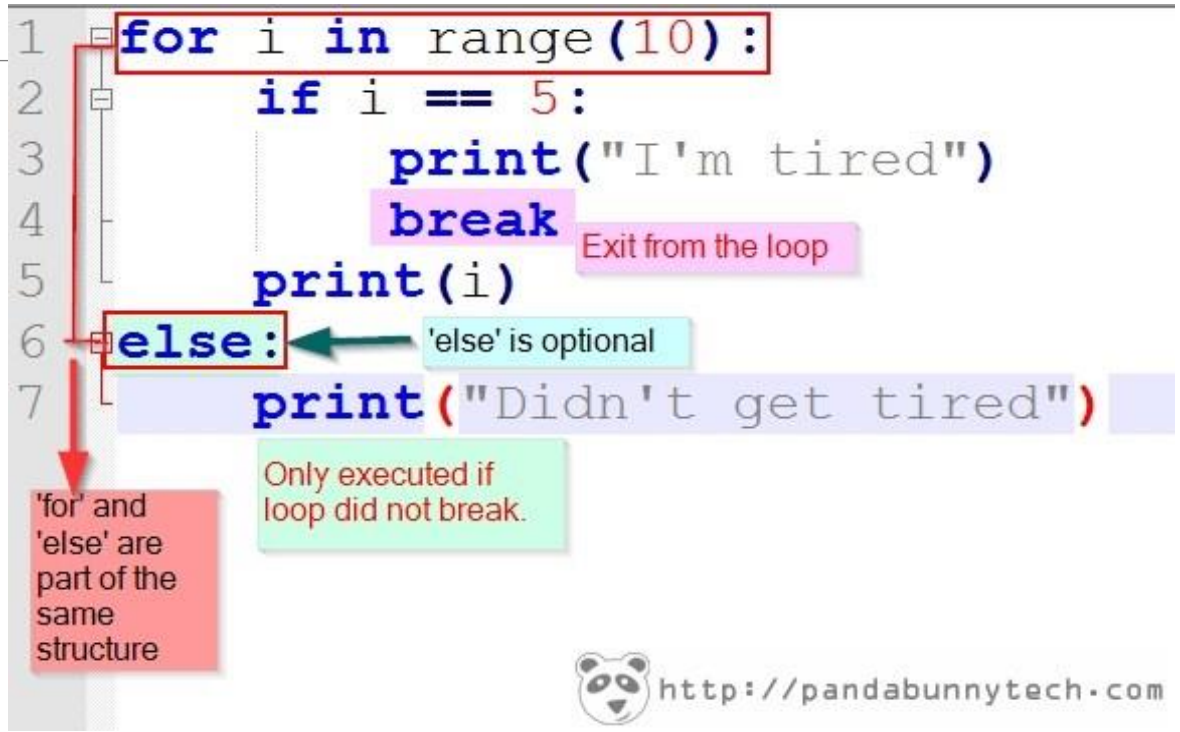


Loops - While

- If you want to execute more than one statement inside one while, you must (as with if) indent all the instructions in the same way;
- An instruction or set of instructions executed inside the while loop is called the loop's body;
- If the condition is False (equal to zero) as early as when it is tested for the first time, the body is not executed even once (note the analogy of not having to do anything if there is nothing to do);
- The body should be able to change the condition's value, because if the condition is True at the beginning, the body might run continuously to infinity - notice that doing a thing usually decreases the number of things to do).

Loops - For

- The **for** loop



Loops - break and continue

- **break** - exits the loop immediately, and unconditionally ends the loop's operation; the program begins to execute the nearest instruction after the loop's body;
- **continue** - behaves as if the program has suddenly reached the end of the body; the next turn is started and the condition expression is tested immediately.
- Both these words are keywords.

Conditional statements

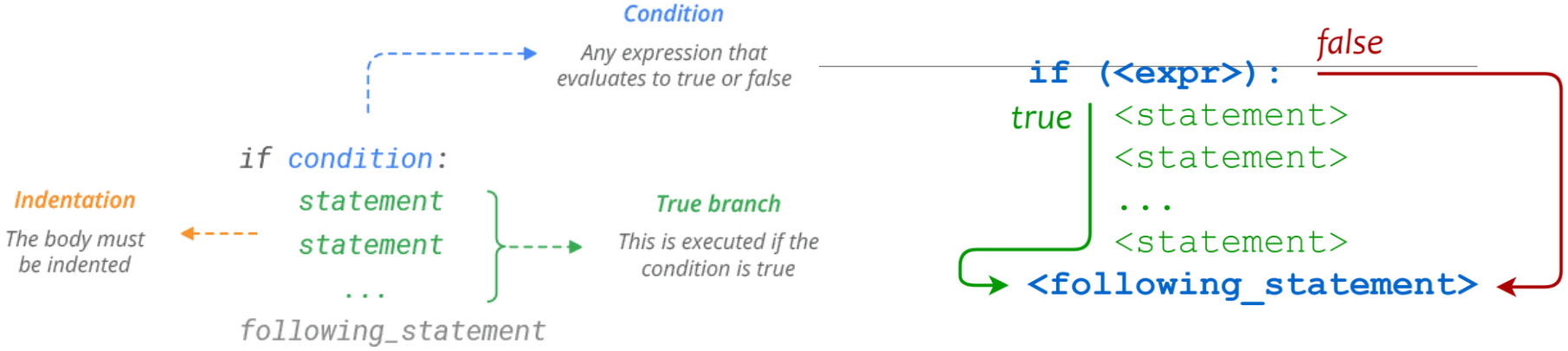
SECTION 5

Conditional execution

- A mechanism which will allow you to do something if a condition is met, and not do it if it isn't

- **How does conditional statement work?**
  - ✓ If the true_or_not expression represents the truth (i.e., its value is not equal to zero), the indented statement(s) will be executed;
  - ✓ if the true_or_not expression does not represent the truth (i.e., its value is equal to zero), the indented statement(s) will be omitted (ignored), and the next executed instruction will be the one after the original indentation level.

Conditional if statement

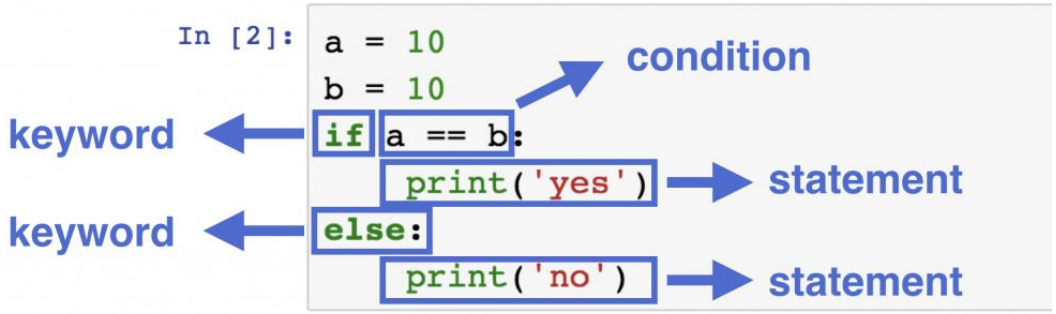



```
if sheep_counter >= 120: # evaluate a test expression
    sleep_and_dream() # execute if test expression is True
```

Conditional if else statement

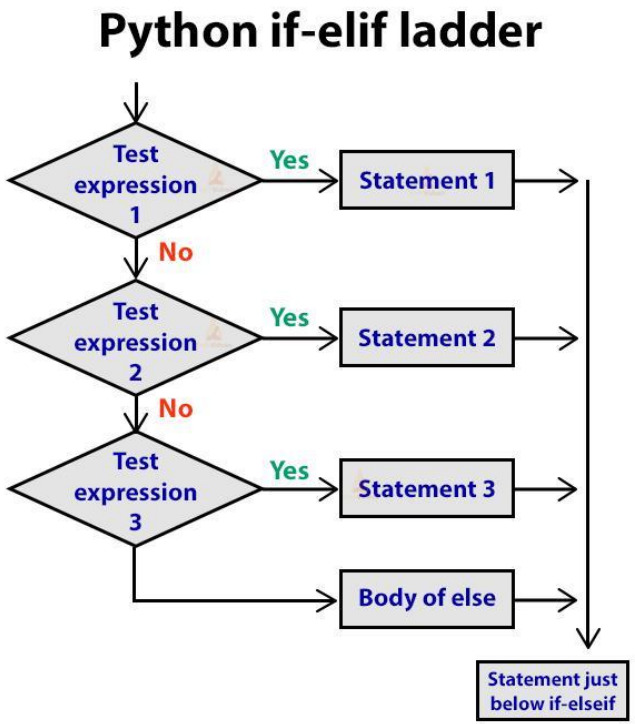
The **if-else** execution goes as follows:

- if the condition evaluates to True (its value is not equal to zero), the perform_if_condition_true statement is executed, and the conditional statement comes to an end;
- if the condition evaluates to False (it is equal to zero), the perform_if_condition_false statement is executed, and the conditional statement comes to an end.



Conditional if elif else statement

- **elif** is used to check more than just one condition, and to stop when the first statement which is true is found.
- **Some additional attention has to be paid in this case:**
  - ✓ you mustn't use else without a preceding if;
  - ✓ else is always the last branch of the cascade, regardless of whether you've used elif or not;
  - ✓ else is an optional part of the cascade, and may be omitted;
  - ✓ if there is an else branch in the cascade, only one of all the branches is executed;
  - ✓ if there is no else branch, it's possible that none of the available branches is executed.



Strings

SECTION 6

Strings Operators

- **CONCATENATION**
  - ✓ The + (plus) sign, when applied to two strings, becomes a concatenation operator
  - ✓ The concatenation operator is not commutative, i.e., "ab" + "ba" is not the same as "ba" + "ab".
- **REPLICATION**
  - ✓ The * (asterisk) sign, when applied to a string and number (or a number and string, as it remains commutative in this position) becomes a replication operator

Strings

Convert a number into a string using str()

The print() function sends data to the console

The input() function gets data from the console.

The result of the input() function is a string

Python methods demo in this notebook:

https://colab.research.google.com/drive/1HndehE-LIw33dANz_2kIuydmsivy6jsW

References

- Main documents are used to develop this material

  https://edube.org/study/pe1

  https://python-tricks.com/

  https://realpython.com/

  https://www.w3schools.com/python/

- Further reading documents

  Guttag, John. *Introduction to Computation and Programming Using Python: With Application to Understanding Data Second Edition*. MIT Press, 2016. ISBN: 9780262529624.

Thank you